ECE 553/653 Neural Networks — Project 2: Q-Learning and Deep Q-Network

Name: YourFirstName YourLastName Date: November 10, 2026

Abstract — An agent must learn to walk from the top-left to the bottom-right corner of a 5×5 maze without hitting walls; the best possible episode reward is 3, earned by a shortest path of 8 moves. We solved the task with tabular Q-Learning and with a Deep Q-Network (DQN) that uses experience replay and a target network, after fixing a bug that made the original target network share its weights with the main network. A one-at-a-time hyperparameter study with 3 seeds per setting showed that the starting exploration rate matters most: lowering it from 0.9 to 0.1 cut the episodes needed to learn from 367.3 to 70.3 for Q-Learning and from 320.3 to 53.3 for DQN. With the best settings, Q-Learning learned in 25.3 ± 2.1 episodes and DQN in 27.0 ± 17.0 episodes, and the greedy agents of both methods walked an optimal 8-move path on every seed. Both methods find the optimal route here, but a DQN step cost about 106× more computing time than a table step, so the table is the better tool for this small maze, while only the network can scale to problems too large for a table.

1. Introduction and Environment

Reinforcement learning (RL) trains an agent by trial and error. In each state *s* the agent takes an action *a*, receives a reward *r* and moves to a new state *s'*; its goal is to collect as much (discounted) reward as possible [1]. In this project the agent is a robot in a 5×5 grid maze (Figure 1). The environment is:

- **Positions and moves.** A position is [x, y]: x is the column and y is the row. "Up" is y−1, "down" is y+1, "left" is x−1 and "right" is x+1. The start is [0, 0] (top-left) and the goal is [4, 4] (bottom-right).
- **Walls.** The program draws 4 random wall cells (x and y between 1 and 4) with the fixed seed 26. Two of the draws land on the same cell, so the maze actually has **3 distinct walls**: (2,3), (3,1), (4,2). We kept this maze unchanged.
- **States and actions.** Each of the 25 cells is one state, numbered s = 5x + y (s0 to s24). There are 4 actions: 0 = up, 1 = right, 2 = down, 3 = left.
- **Rewards.** +10 for stepping onto the goal (the episode ends), −10 for moving into a wall (the agent stays where it is), and −1 for every other move. Bumping into the outer edge also leaves the agent in place, but costs only −1. An episode also ends after 499 moves.

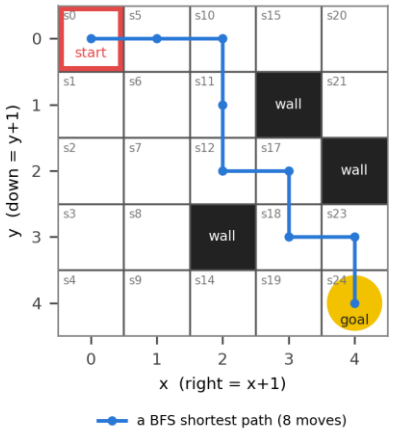


Figure 1. The maze (seed 26). Grey labels are state numbers s = 5x + y; black cells are walls; red square: start; yellow circle: goal. Blue: one of the 17 shortest paths (8 moves).

Best possible reward. A breadth-first search (BFS) from the start shows that the shortest route has L = 8 moves, and that 17 different routes share this length (one is drawn in Figure 1). The best episode reward is therefore seven ordinary moves at −1 plus the final move onto the goal at +10:

$$R^* = 10 - (L - 1) = 10 - 7 = 3 \quad (1)$$

We use R* = 3 as the "perfect score" line in all plots, and we say an agent has **learned** at the first episode where its 10-episode average reward is within 1 of it (at least 2).

2. Action Selection Mechanism

Both agents choose actions with the **ε-greedy** rule (Task I). With probability ε the agent takes a uniformly random action (exploration); otherwise it takes the action with the highest current Q-value (exploitation). Written as the probability of choosing action a in state s:

$$\pi(a \mid s) = \frac{\varepsilon}{4} + (1 - \varepsilon)\frac{\mathbf{1}[a \in A^*(s)]}{|A^*(s)|}, A^*(s) = \operatorname{argmax}_{a'} Q(s, a') \quad (2)$$

Here A*(s) is the set of actions that share the highest Q-value, and 1[·] is 1 if a belongs to that set and 0 otherwise. Every action keeps a probability of at least ε/4, so no action is ever ruled out.

Random tie-breaking. The Q-table starts with all zeros, so at first every action ties. A plain arg max would then always return the first index, "up". From the start cell "up" only bumps into the top edge, so an almost-greedy agent could waste whole

episodes standing still. We therefore choose at random among all actions that share the highest value. Together with the −1 step cost this gives a built-in kind of exploration: once an action has been tried its value drops below zero, so the untried actions (still at 0) win the next arg max. The DQN uses the same rule on the four outputs of its network, where exact ties practically never happen.

Shrinking ε. After every episode ε is multiplied by 0.995, and it never goes below 0.01:

$$\varepsilon_{k+1} = \max(0.01, 0.995\varepsilon_k) \quad (3)$$

Starting from $\varepsilon_0 = 0.9$, ε falls to 0.45 after 139 episodes and to 0.07 after 500 episodes, so it never reaches the floor in our 500-episode runs. Starting from $\varepsilon_0 = 0.1$, it reaches the floor of 0.01 after about 460 episodes. **From exploring to exploiting:** at the start the Q-values mean nothing, so random moves are the only way to find out where the walls and the goal are. As ε shrinks, the agent trusts what it has learned more and more, and its episodes get close to the shortest path. Q-Learning is *off-policy*: it learns the values of the greedy policy even while it explores, so exploration lowers the reward collected during training but does not change what is finally learned. Sections 5.3 and 5.6 show that on this small maze a low starting value ($\varepsilon_0 = 0.1$) is enough and much faster.

3. Algorithm Design

3.1 Q-table update (Task II)

The Q-table holds one number for every state–action pair (25 × 4 = 100 numbers), all starting at 0. After every move (s, a, r, s′) the entry Q(s, a) is moved a fraction α of the way toward a one-step target y [2], where d = 1 if s′ is the goal and d = 0 otherwise:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)], y = r + \gamma(1-d)\max_{a'} Q(s',a') \quad (4)$$

- **α (learning rate)** sets how far the entry moves toward the target: α = 1 would simply replace it, α = 0.01 moves it only 1 % of the way. This maze is deterministic, so the target is not noisy and a large α is safe (Section 5.1).
- **γ (discount factor)** sets how much future reward counts. With γ = 0.99 the +10 at the goal is still worth $10 \times 0.99^7 \approx 9.3$ when seen from the start, so the way to the goal is clearly marked; with γ = 0.5 it is worth only 0.08.
- **Goal state.** Reaching the goal ends the episode, so nothing comes after it: d = 1 and the target is just r = +10. Without this rule the update would add γ times the values stored for the goal cell, which are never trained.
- **Walls.** A move into a wall gives r = −10 and s′ = s, so actions that point into walls quickly get low values.

3.2 DQN update (Tasks III and IV)

The DQN replaces the table with a neural network Q(s, ·; θ) with weights θ that outputs the four action values of a state [3]:

- **Network (Task III).** The input is a one-hot vector of length 25 (a 1 at the agent's cell). It passes through two fully connected hidden layers of 64 units with ReLU activations and a linear output layer with 4 units (25 → 64 → 64 → 4), which gives 6,084 trainable weights and biases. The hidden size can be changed from main.py (Section 5.9).
- **Replay buffer.** Every move is stored (capacity 1,000; the oldest moves are dropped first). Once more than B = 64 moves are stored, a random mini-batch of B moves is drawn after every move and used for one gradient step. Random sampling breaks the strong link between consecutive moves and reuses each experience many times.
- **Target network.** A second copy of the network, with weights θ^-, computes the targets. Its weights are copied from the main network every C = 10 learning steps, so the targets stay fixed for a while instead of moving with every update.
- **Expected Q-values and loss (Task IV).** For each sampled move the target is computed with the target network (d = 1 if s′ is the goal, otherwise 0). The target is detached, so gradients flow only through the prediction $Q(s_i, a_i; \theta)$. The loss is the mean squared error over the batch, minimized with the Adam optimizer:

$$y_i = r_i + \gamma(1-d_i)\max_{a'} Q_{\theta^-}(s_i', a') \quad (5)$$

$$L(\theta) = \frac{1}{B}\sum_{i=1}^{B} (y_i - Q_\theta(s_i, a_i))^2 \quad (6)$$

Bug fixes. (1) In QLearning.py the line that chooses the action had the wrong indentation, so the file could not even be loaded; it was fixed while writing Task I. (2) More importantly, the provided DQN created the target network with a *shallow copy* (copy.copy). A shallow copy of a PyTorch module is a new module object that still holds the very same weight tensors, and we confirmed that the weights of both networks were the same objects. Every gradient step on the main network therefore also changed the "target", and the line meant to freeze the target only set an unused attribute, so the program effectively had no target network. We replaced it with a *deep copy* (copy.deepcopy), which gives the target network its own weights, and turned off gradients for each of its parameters. Now the target changes only when it is synchronized every C learning steps.

3.3 Differences between the two update mechanisms

Table 1 sums up the differences. **What one update changes:** a table update changes exactly one number, Q(s, a), while a DQN update changes shared weights, so the values of many states move at once. This is what lets a network *generalize* to states it has rarely seen, as long as similar states get similar inputs (with our one-hot input the sharing happens only in the hidden layers). **Data per update:** Q-Learning learns from the single latest move and then forgets it; DQN learns from a random mini-batch of 64 stored moves, which reuses experience and averages out the gradient. **Stability:** for a finite problem like this one the table update converges to the optimal values under standard conditions [2], but a network chases a target computed with its own weights, which can make training oscillate or diverge; the replay buffer and the target network keep it stable [3]. **Speed and memory:** with the best settings one Q-Learning step took about 9 µs and one DQN step (action choice, mini-batch forward/backward pass and Adam update) about 0.95 ms, roughly 106 times longer; the table stores 100 numbers, while the DQN stores 6,084 weights twice (main and target network) plus the optimizer state and up to 1,000 moves. **Bigger problems:** a table needs one entry per state, which is impossible when states are very many or continuous (images, joint angles), while the size of a network does not grow with the number of states. DQN is therefore the method that can scale, provided its input lets it generalize (see EC5 in Section 6).

Table 1. Key differences between the Q-table update and the DQN update.

	Q-Learning (table)	DQN (neural network)
How values are stored	100 numbers, one per (state, action)	Network 25→64→64→4 (6,084 parameters)
What one update changes	One entry	All shared weights (values of many states move)
Data per update	The latest move only	Random mini-batch of 64 stored moves
Target	$r + \gamma \max Q(s', \cdot)$ from the same table	$r + \gamma \max Q(s', \cdot)$ from a separate target network
Extra stability tools	None needed	Replay buffer, target network, detached target
Measured time per step (best settings)	9 µs	0.95 ms
Generalization	None (each cell learned separately)	Possible through shared weights
Best suited for	Small, discrete problems	Large or continuous state spaces

4. Experimental Results

With the best settings of Section 5.10 we ran main.py once for each method and then one test episode with $\varepsilon = 0$ (no random moves). Both agents reached the goal in 8 moves with a reward of 3, the best possible result (Figure 2). Figure 3 shows the training rewards of the same runs.

Table 2. Learned paths compared with the BFS shortest path.

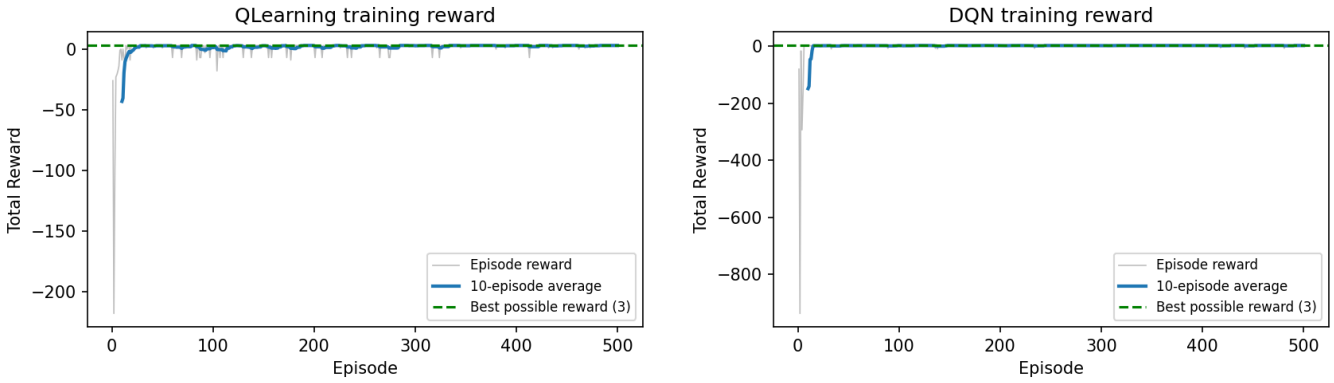
Path	Cells visited (x, y)	Moves	Reward
BFS (one shortest path)	(0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (3,2) → (3,3) → (4,3) → (4,4)	8	3
Q-Learning, greedy	(0,0) → (0,1) → (1,1) → (1,2) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4)	8	3
DQN, greedy	(0,0) → (1,0) → (1,1) → (1,2) → (1,3) → (1,4) → (2,4) → (3,4) → (4,4)	8	3

All three paths have 8 moves. The learned paths differ from the BFS path and from each other, which is expected because 17 different paths are equally short; each agent found one of them.



(a) Q-Learning (b) DQN

Figure 2. Game window after the greedy test episode. The red square (the agent) is on the goal; the blue line is the path it walked.



(a) Q-Learning (b) DQN

Figure 3. Training reward in the main.py runs with the best settings (grey: each episode, blue: 10-episode average, green dashed line: best possible reward 3).

Both runs print the mean reward of the last 5 episodes every 5 episodes. Q-Learning's mean first reaches 2 or more at episode 25. The DQN run has a few long episodes at the very start (worst 5-episode mean reward -283), but it also gets close to the best reward by about episode 10 and stays there; the small dips later come from the remaining random moves.

Table 3. Q-Learning vs DQN with the best settings (mean ± standard deviation over seeds 0, 1, 2).

Metric (best settings, 3 seeds)	Q-Learning	DQN
Settings	$\alpha = 0.5$, $\gamma = 0.99$, $\varepsilon_0 = 0.1$	lr = 1e-3, $\gamma = 0.99$, $\varepsilon_0 = 0.1$, batch 64, C = 10, hidden 64
Episodes to learn	25.3 ± 2.1	27.0 ± 17.0
Average reward, last 50 episodes	2.98 ± 0.02	2.87 ± 0.10
Training time, 500 episodes (s)	0.04 ± 0.00	10.78 ± 8.44
Environment steps, 500 episodes	4,471	11,024
Time per step	8.9 ± 0.1 µs	0.95 ± 0.04 ms
Greedy test: seeds reaching the goal	3/3 (8 moves)	3/3 (8 moves)

Both methods solve the maze on every seed and reach almost the same final reward. Q-Learning learned faster and much more consistently, and it needed about 106× less time per step. DQN's large spread comes from seed 0, where the untrained network got stuck in the first episodes and learning took 51 episodes, compared with 15 and 15 for the other two seeds.

5. Hyperparameter Analysis

Setup. We changed one setting at a time from a fixed starting point (Q-Learning: $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon_0 = 0.9$; DQN: learning rate 1e-4, $\gamma = 0.99$, $\varepsilon_0 = 0.9$, batch 64, buffer 1,000, target update every C = 10 steps, hidden size 64). Each setting was trained for 500 episodes with seeds 0, 1 and 2, which are set after the maze is built so the maze never changes; drawing was turned off. We report the mean ± standard deviation over the 3 seeds of (a) the average reward of the last 50 episodes, (b) the episodes needed to learn, with the number of seeds that learned if fewer than 3, (c) the training time and (d) a greedy test ($\varepsilon = 0$, at most 100 moves): how many seeds reached the goal and in how many moves. The total number of environment steps is also listed, because it explains most of the time differences. Runs were executed four at a time on a laptop CPU, so times are comparable with each other but slightly noisy. The starting-point run is shared by all sweeps of a method. Every figure shows the 10-episode average reward (mean ± std band, y-axis cut at −60), the last-50 reward and the training time.

Note on the last-50 reward. With $\varepsilon_0 = 0.9$, ε is still 0.09–0.07 in the last 50 episodes, so this number also counts random moves. Differences of about 0.5 are within the seed-to-seed spread, so the episodes to learn and the greedy test are the more reliable signals.

5.1 Q-Learning: learning rate α

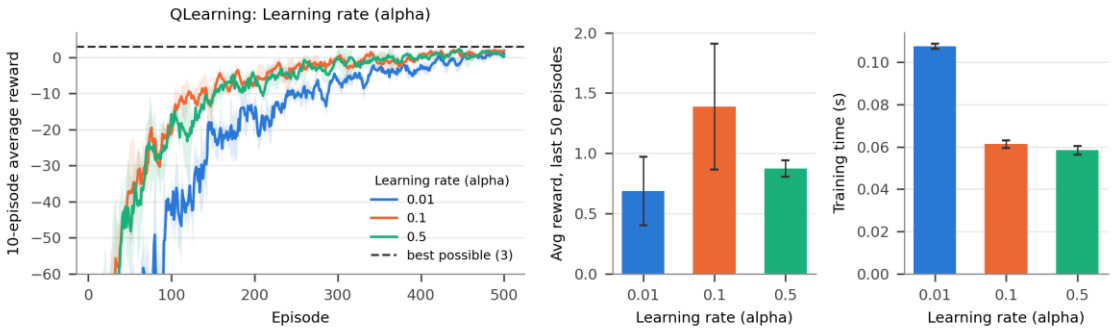


Figure 4. Q-Learning, learning rate α. Left: learning curves (mean ± std over 3 seeds); middle: average reward of the last 50 episodes; right: training time.

Table 4. Q-Learning: effect of the learning rate (alpha) (mean ± std over 3 seeds).

Learning rate (alpha)	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
0.01	0.69 ± 0.28	475.5 ± 18.5 (2/3 learned)	0.11 ± 0.00	13,972	2/3 (8 moves)
0.1	1.39 ± 0.52	367.3 ± 32.8	0.06 ± 0.00	8,022	3/3 (8 moves)
0.5	0.87 ± 0.07	323.3 ± 22.4	0.06 ± 0.00	7,483	3/3 (8 moves)

Why. With α = 0.01 each update moves a Q-value only 1 % of the way to its target, so the information "the goal is here" travels back toward the start very slowly. This was the only Q-Learning setting that failed: one seed never met the learning criterion and another seed's greedy agent did not reach the goal within 100 moves. Slow learning also means long, wandering episodes (13,972 steps), which made it the slowest setting (0.11 ± 0.00 s, still tiny). A large α = 0.5 learned fastest (323.3 episodes) because the maze is deterministic: the target has no noise that a small α would need to average out. Its last-50 reward (0.87 ± 0.07) is lower than for α = 0.1 (1.39 ± 0.52), but this difference is about the size of the seed-to-seed spread and both reach the goal optimally on every seed.

5.2 Q-Learning: discount factor γ

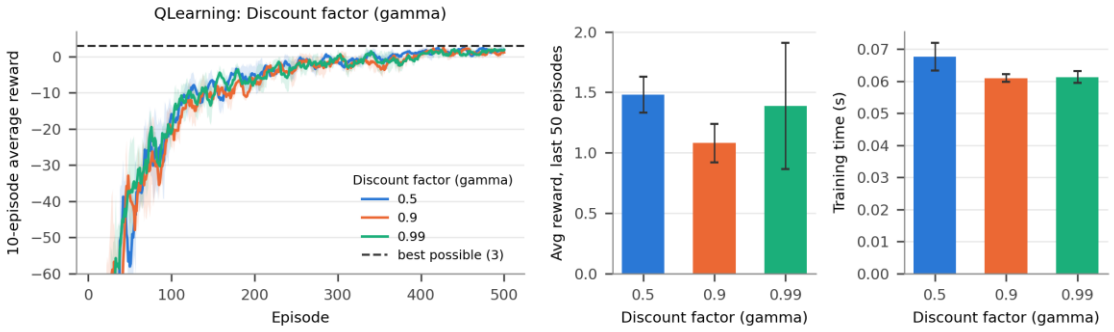


Figure 5. Q-Learning, discount factor γ (same layout as Figure 4).

Table 5. Q-Learning: effect of the discount factor (gamma) (mean ± std over 3 seeds).

Discount factor (gamma)	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
0.5	1.48 ± 0.15	354.0 ± 21.5	0.07 ± 0.00	8,322	3/3 (8 moves)
0.9	1.08 ± 0.16	367.3 ± 42.7	0.06 ± 0.00	8,098	3/3 (8 moves)
0.99	1.39 ± 0.52	367.3 ± 32.8	0.06 ± 0.00	8,022	3/3 (8 moves)

Why. γ had little effect here: all three values learned in 354.0–367.3 episodes, all times were 0.06–0.07 s and every greedy agent was optimal. The maze is small and deterministic, and the −1 step cost already rewards short paths, so even γ = 0.5 orders the actions correctly in a table. The catch is that with γ = 0.5 the goal reward seen from the start is only $10 \times 0.5^7 \approx 0.08$, so values far from the goal become almost flat. That is harmless for an exact table but risky for longer paths or for a network, whose small errors could then flip the best action (see Section 5.5). We therefore kept γ = 0.99.

5.3 Q-Learning: starting exploration rate ε_0

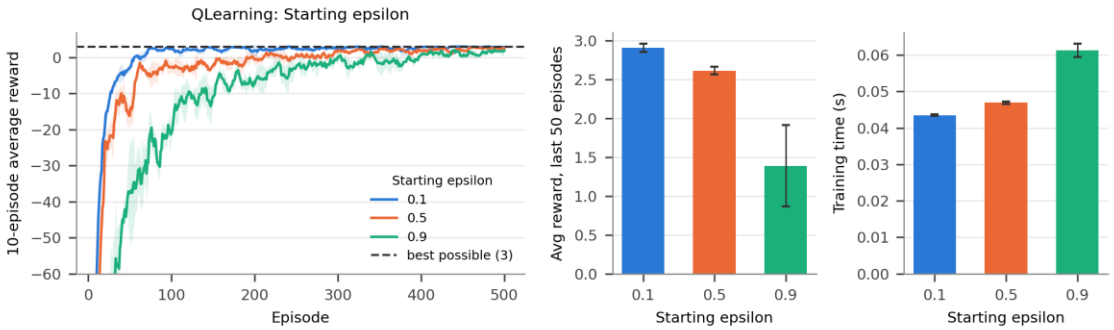


Figure 6. Q-Learning, starting exploration rate ε_0 (same layout as Figure 4).

Table 6. Q-Learning: effect of the starting epsilon (mean ± std over 3 seeds).

Starting epsilon	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
0.1	2.91 ± 0.05	70.3 ± 3.1	0.04 ± 0.00	5,049	3/3 (8 moves)
0.5	2.61 ± 0.05	231.0 ± 18.1	0.05 ± 0.00	5,843	3/3 (8 moves)
0.9	1.39 ± 0.52	367.3 ± 32.8	0.06 ± 0.00	8,022	3/3 (8 moves)

Why. This was by far the strongest effect. With $\varepsilon_0 = 0.1$ the agent learned in 70.3 ± 3.1 episodes, about 5× faster than with $\varepsilon_0 = 0.9$ (367.3), and its last-50 reward was almost perfect (2.91 ± 0.05). The maze has only 25 cells, and the zero-initialised table with random tie-breaking already explores untried actions on its own (Section 2), so little random exploration is needed. With $\varepsilon_0 = 0.9$ the agent acts randomly 90 % of the time at first, and ε is still about 0.07 after 500 episodes, which keeps adding detours and wall hits. Less wandering also means fewer steps (5,049 vs 8,022) and the shortest time. In a larger or trickier environment such a small ε could miss better routes, so this result is specific to small mazes.

5.4 DQN: learning rate

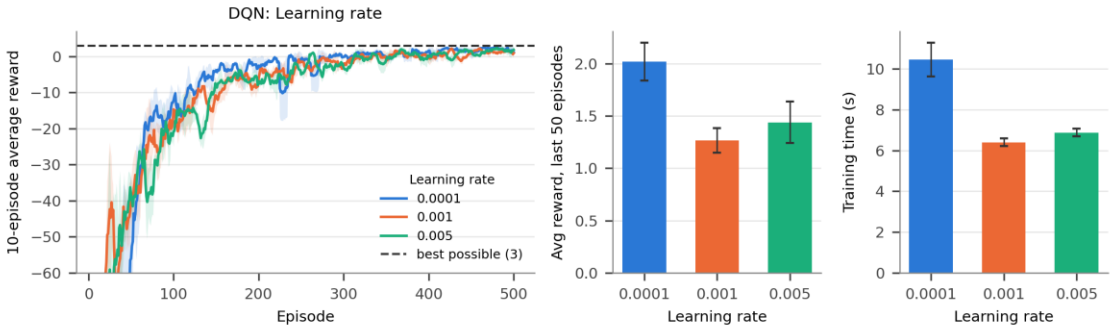


Figure 7. DQN, learning rate (same layout as Figure 4).

Table 7. DQN: effect of the learning rate (mean ± std over 3 seeds).

Learning rate	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
1e-4	2.02 ± 0.18	320.3 ± 40.4	10.46 ± 0.83	12,133	3/3 (8 moves)
1e-3	1.27 ± 0.12	313.3 ± 16.8	6.40 ± 0.19	7,379	3/3 (8 moves)
5e-3	1.44 ± 0.20	385.0 ± 53.2	6.88 ± 0.19	7,712	3/3 (8 moves)

Why. All three learning rates reached the criterion on every seed. 1e-3 learned about as fast as 1e-4 (313.3 vs 320.3 episodes) but with fewer steps (7,379 vs 12,133), because the network corrected its bad early estimates sooner and the long early episodes ended earlier. Since every DQN step includes one gradient update, fewer steps means less time (6.40 ± 0.19 s vs 10.46 ± 0.83 s). The largest rate, 5e-3, was slowest to learn (385.0 episodes): its big weight changes make the Q-estimates jump around. The small rate 1e-4 had the best last-50 reward (2.02 ± 0.18); a likely reason is that larger steps keep changing the values of rarely visited cells, so the moves after a random detour are less reliable. The final greedy agents were optimal for every rate. Section 5.10 shows that 1e-3 is the better choice once $\varepsilon_0 = 0.1$.

5.5 DQN: discount factor γ

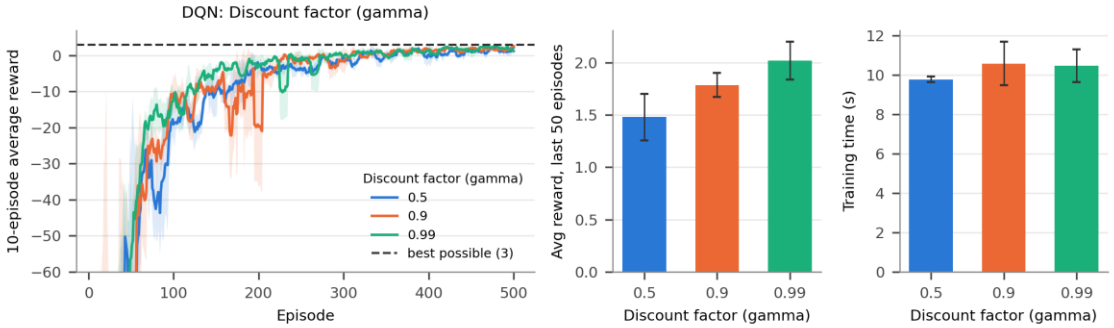


Figure 8. DQN, discount factor γ (same layout as Figure 4).

Table 8. DQN: effect of the discount factor (gamma) (mean ± std over 3 seeds).

Discount factor (gamma)	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
0.5	1.48 ± 0.22	327.7 ± 3.3	9.77 ± 0.15	10,864	3/3 (8 moves)
0.9	1.79 ± 0.11	336.7 ± 26.0	10.58 ± 1.11	11,452	3/3 (8 moves)
0.99	2.02 ± 0.18	320.3 ± 40.4	10.46 ± 0.83	12,133	3/3 (8 moves)

Why. The learning speed was similar (320.3–336.7 episodes), but the last-50 reward rose steadily with γ: 1.48 ± 0.22 → 1.79 ± 0.11 → 2.02 ± 0.18. Unlike the table, the network only approximates the values. With a small γ the values of neighbouring cells far from the goal differ very little, so small approximation errors are more likely to pick a wrong action in cells the agent seldom visits; with γ = 0.99 the differences are larger and easier to fit. Times were similar (9.8–10.6 s) and followed the number of steps.

5.6 DQN: starting exploration rate ε_0

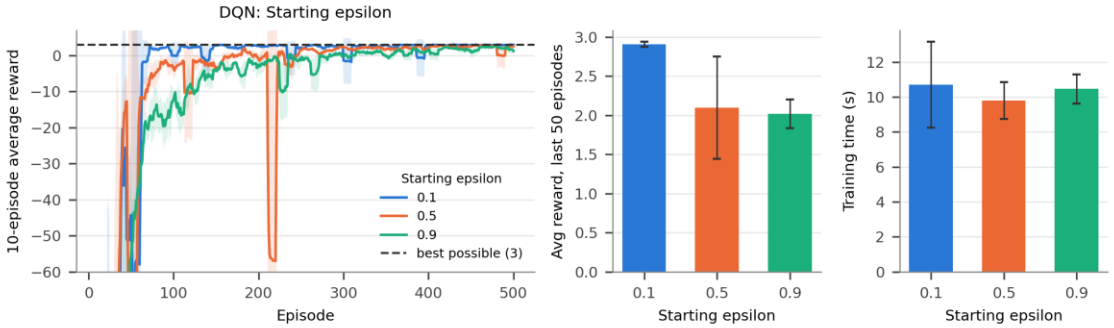


Figure 9. DQN, starting exploration rate ε_0 (same layout as Figure 4).

Table 9. DQN: effect of the starting epsilon (mean ± std over 3 seeds).

Starting epsilon	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
0.1	2.91 ± 0.03	53.3 ± 21.0	10.70 ± 2.46	11,109	3/3 (8 moves)
0.5	2.10 ± 0.65	195.7 ± 20.0	9.78 ± 1.05	10,454	3/3 (8 moves)
0.9	2.02 ± 0.18	320.3 ± 40.4	10.46 ± 0.83	12,133	3/3 (8 moves)

Why. As for Q-Learning, less exploration was much better: 53.3 ± 21.0 episodes with ε_0 = 0.1 versus 195.7 with 0.5 and 320.3 with 0.9, and a last-50 reward of 2.91 ± 0.03. Unlike Q-Learning, however, the time did not drop (10.70 ± 2.46 s vs 10.46 ± 0.83 s). The randomly initialised network has no ties, so with ε_0 = 0.1 it simply follows its arbitrary greedy action and several of the first episodes run all the way to the 499-move limit before the network has learned enough. These long early episodes cost about as many steps (11,109 vs 12,133) as the random wandering with ε_0 = 0.9, and they also explain the larger spread of the learning speed (±21).

5.7 DQN: target network update frequency

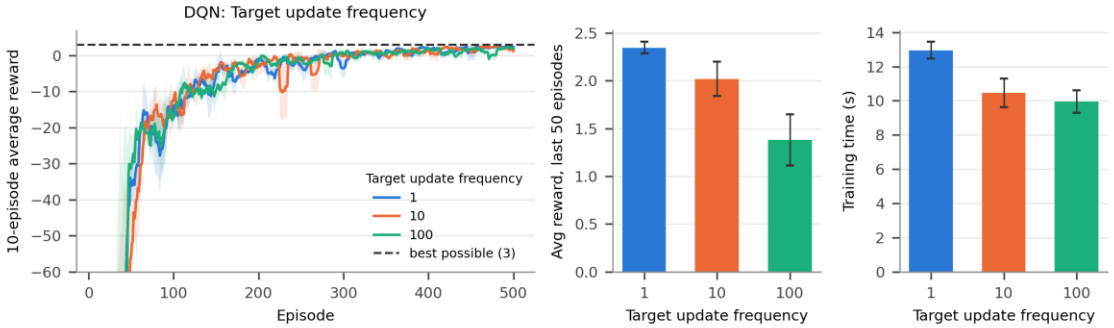


Figure 10. DQN, target network update frequency C (same layout as Figure 4).

Table 10. DQN: effect of the target update frequency (mean ± std over 3 seeds).

Target update frequency	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
1	2.35 ± 0.06	343.3 ± 33.9	12.95 ± 0.50	13,312	3/3 (8 moves)
10	2.02 ± 0.18	320.3 ± 40.4	10.46 ± 0.83	12,133	3/3 (8 moves)
100	1.38 ± 0.27	297.0 ± 5.4	9.95 ± 0.66	11,241	3/3 (8 moves)

Why. Copying the weights after every learning step (C = 1) is the same as having no separate target network. It gave the best last-50 reward (2.35 ± 0.06), C = 10 was in the middle (2.02 ± 0.18) and C = 100 was lowest (1.38 ± 0.27). Fresh targets pass the latest knowledge on at once, while targets that are up to 100 steps old lag behind the network. The learning speeds (297.0–343.3 episodes) are within about one standard deviation of each other. On a problem this small and this deterministic, the moving-target instability that the target network prevents hardly appears; on larger problems the slower target is what keeps DQN stable [3]. C = 1 was also the slowest (12.95 ± 0.50 s), because it copies all weights at every step and took the most steps.

5.8 DQN: batch size

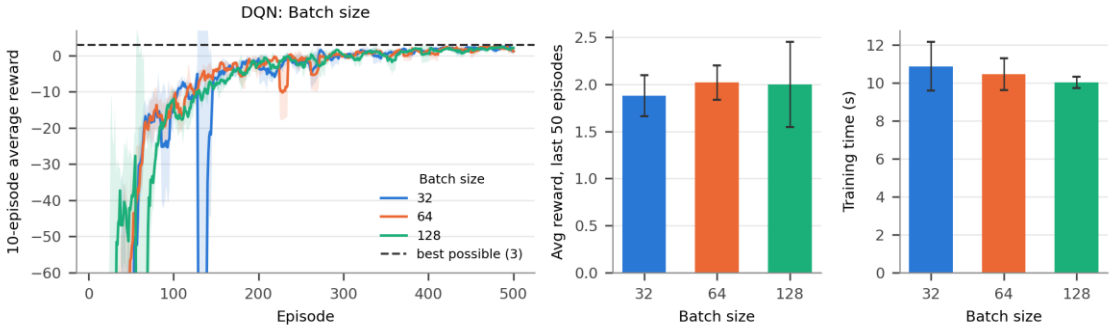


Figure 11. DQN, batch size (same layout as Figure 4).

Table 11. DQN: effect of the batch size (mean ± std over 3 seeds).

Batch size	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
32	1.88 ± 0.22	284.7 ± 17.2	10.87 ± 1.29	13,307	3/3 (8 moves)
64	2.02 ± 0.18	320.3 ± 40.4	10.46 ± 0.83	12,133	3/3 (8 moves)
128	2.00 ± 0.45	344.3 ± 40.1	10.02 ± 0.30	10,449	3/3 (8 moves)

Why. The batch size changed little. Batch 32 learned a bit faster (284.7 vs 320.3 and 344.3 episodes), partly because learning starts as soon as the buffer holds more moves than one batch (33 instead of 65 or 129). Larger batches give smoother gradient estimates, and 64 and 128 had nearly the same last-50 reward (2.02 ± 0.18 and 2.00 ± 0.45). The time was almost the same (10.0–10.9 s): for such a small network, the fixed cost of each update matters more than the number of samples in it, so the time follows the number of steps rather than the batch size.

5.9 DQN: hidden layer size

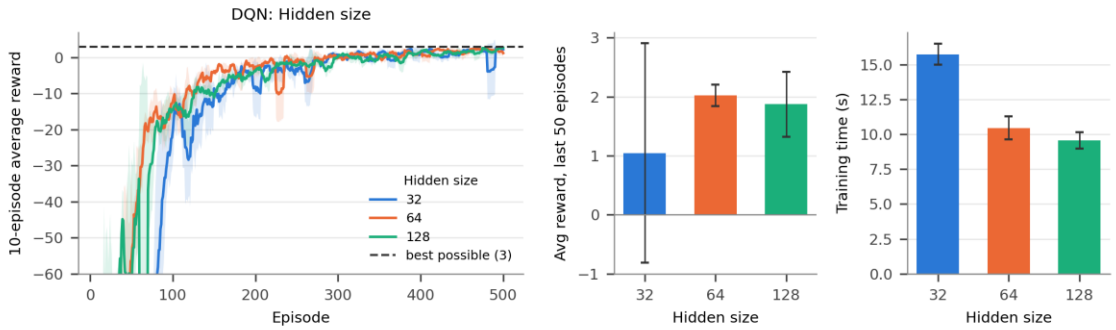


Figure 12. DQN, hidden layer size (same layout as Figure 4).

Table 12. DQN: effect of the hidden size (mean ± std over 3 seeds).

Hidden size	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
32	1.05 ± 1.86	344.0 ± 24.8	15.75 ± 0.75	19,109	3/3 (8 moves)
64	2.02 ± 0.18	320.3 ± 40.4	10.46 ± 0.83	12,133	3/3 (8 moves)
128	1.87 ± 0.55	335.3 ± 45.5	9.58 ± 0.59	9,408	3/3 (8 moves)

Why. 64 units gave the best last-50 reward (2.02 ± 0.18) and the fastest learning (320.3 episodes). 32 units are enough in principle (there are only 100 values to represent), but training was less reliable: one seed had a poor final phase (last-50 reward -1.58), which explains the huge spread (1.05 ± 1.86). The small network also needed the most steps (19,109) and therefore the most time (15.75 ± 0.75 s). 128 units learned well and used the fewest steps (9,408, 9.58 ± 0.59 s): a wider network fits the values in fewer updates, and at this size the extra computation per step is negligible. Every greedy agent was optimal.

5.10 Combining the settings: the best configuration

A one-at-a-time study cannot see interactions between settings. We therefore took the clear winner (ε_0 = 0.1 for both methods) and trained it together with each candidate for the one setting whose sweep was mixed: α for Q-Learning and the learning rate for DQN (Table 13). All other settings stayed at their starting values, since their sweeps either favoured the starting value (γ = 0.99, hidden size 64) or showed only small, mixed differences (batch size, target update frequency).

Table 13. Combined settings (mean ± std over 3 seeds). Bold rows are the chosen best settings.

Combination (3 seeds)	Avg reward, last 50 ep.	Episodes to learn	Time (s)	Env. steps	Greedy test: goal
Q-Learning: ε_0 0.1, α 0.1	2.91 ± 0.05	70.3 ± 3.1	0.04 ± 0.00	5,049	3/3 (8 moves)
Q-Learning: ε_0 0.1, α 0.5	**2.98 ± 0.02**	**25.3 ± 2.1**	**0.04 ± 0.00**	**4,471**	**3/3 (8 moves)**
DQN: ε_0 0.1, lr 1e-4	2.91 ± 0.03	53.3 ± 21.0	10.70 ± 2.46	11,109	3/3 (8 moves)
DQN: ε_0 0.1, lr 1e-3	**2.87 ± 0.10**	**27.0 ± 17.0**	**10.78 ± 8.44**	**11,024**	**3/3 (8 moves)**

For Q-Learning, α = 0.5 with ε_0 = 0.1 is clearly best: it learned in 25.3 ± 2.1 episodes (vs 70.3) with a nearly perfect last-50 reward. For DQN, lr = 1e-3 with ε_0 = 0.1 learned in 27.0 episodes on average (vs 53.3), with the same final reward within the spread; on two seeds it needed only 15 episodes, while seed 0 got stuck early and needed 51 episodes and 22,565 steps, which also explains the large time spread. These settings are now the defaults in main.py: **Q-Learning: α = 0.5, γ = 0.99, ε_0 = 0.1**; **DQN: learning rate 1e-3, γ = 0.99, ε_0 = 0.1, batch 64, buffer 1,000, C = 10, hidden size 64**, both with 500 episodes.

6. Extra Credit

All extra-credit code is in extra_credit.py (main.py is unchanged by it). Every experiment uses seeds 0, 1 and 2, set after the maze is built, and the best settings of Section 5.10 unless stated otherwise. All numbers are in extra_credit_summary.csv.

6.1 EC1 — Policy and value maps

What we did. For each trained agent we drew the greedy action (arrow) and the best Q-value, max over a of Q(s, a) (colour and number), in every free cell, next to the true optimal values computed by value iteration on the known maze.

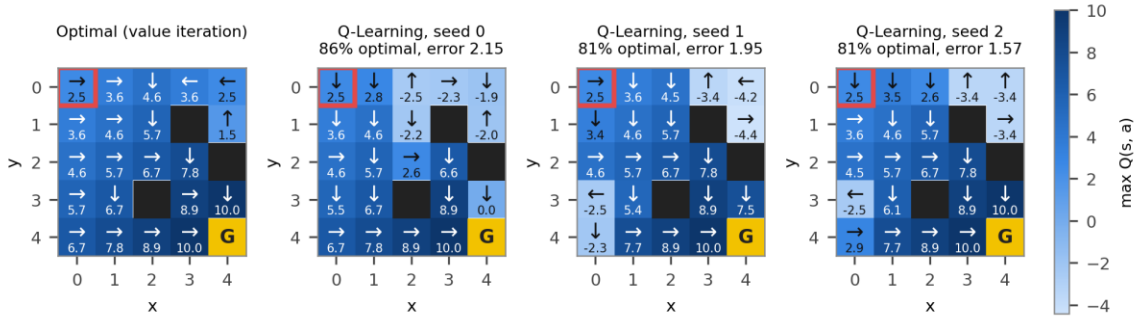


Figure 13. Q-Learning policy and value maps (best settings). Left: optimal values and actions from value iteration (when two actions are equally good, one is shown). Other panels: the learned greedy action and max Q in each cell.

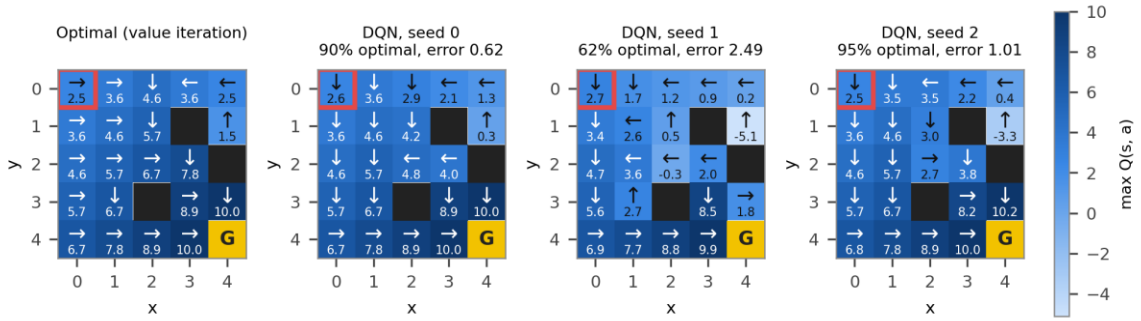


Figure 14. DQN policy and value maps (same layout as Figure 13).

Result. On average 82.5 % of the free cells had an optimal greedy action for both Q-Learning (± 2.2 %) and DQN (± 14.7 %). The mean value error |max Q − V*| was 1.89 ± 0.24 for Q-Learning and 1.37 ± 0.81 for DQN. Along the route each agent really uses, the arrows are optimal and the start value is close to the true V* = 2.5. The wrong arrows sit in cells away from that route, which are rarely visited with $\varepsilon_0 = 0.1$: mainly the top-right corner, and for DQN seed 1 (62 % optimal) also the middle of the maze, which its route along the left and bottom edges never crosses.

What it shows. An agent only knows the part of the maze it has explored. That is enough to walk the best route, but its map of the rest of the maze can be wrong, which is the price of exploring little.

6.2 EC2 — Double DQN

What we did. Double DQN [4] changes only the target: the main network chooses the best next action and the target network gives its value. We compared it with the normal DQN and, every 10 episodes, measured the gap between the predicted Q-values and the true Q* (from value iteration) over all free cells and actions.

$$y_i = r_i + \gamma(1-d_i)Q_{\theta^-}(s_i', \operatorname{argmax}_{a'} Q_\theta(s_i', a')) \tag{7}$$

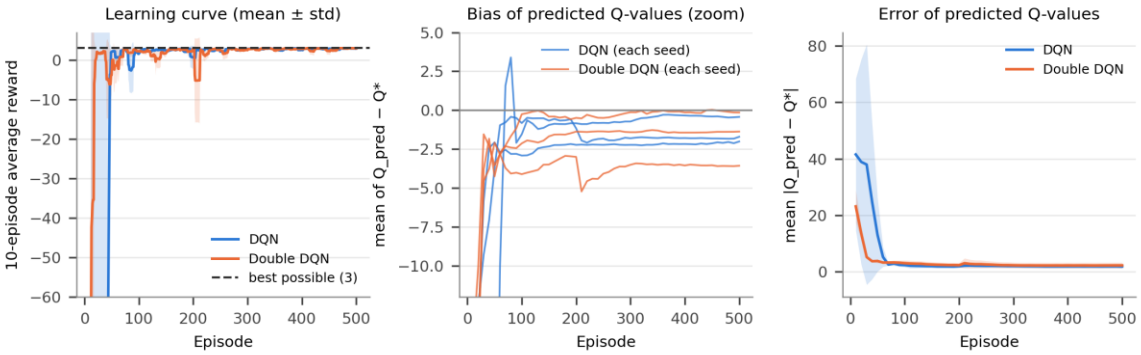


Figure 15. DQN vs Double DQN. Left: learning curves (mean ± std over 3 seeds). Middle: mean (Q_pred − Q*) for each seed, y-axis cut at −12 to show the later part. Right: mean |Q_pred − Q*| (mean ± std).

Table 14. DQN vs Double DQN (mean ± std over 3 seeds).

Metric (3 seeds)	DQN	Double DQN
Episodes to learn	27.0 ± 17.0	21.3 ± 2.4
Average reward, last 50 episodes	2.87 ± 0.10	2.85 ± 0.13
Training time (s)	10.83 ± 8.31	5.38 ± 0.57
Mean |Q_pred − Q*| over training	5.05 ± 2.54	3.26 ± 0.73
Mean |Q_pred − Q*| at the end	1.90 ± 0.62	2.28 ± 1.04
Largest bias (Q_pred − Q*) during training	0.31 ± 2.27	-0.94 ± 0.69
Greedy test: seeds reaching the goal	3/3 (8 moves)	3/3 (8 moves)

Result. Double DQN learned faster and much more consistently (21.3 ± 2.4 vs 27.0 ± 17.0 episodes, so it also needed less time) and its Q-error averaged over training was smaller (3.26 vs 5.05). The final reward and final error were the same within the spread. Over-estimation was small in this maze: the normal DQN's bias went clearly above zero on 1 seed (up to +3.39), while Double DQN's bias never rose above +0.02.

What it shows. Taking the maximum over noisy estimates tends to pick values that are too high. Letting a second network judge the chosen action removes most of this upward bias [4]. Here all early rewards are negative, so both networks start far below Q* and the bias is mostly negative; the benefit is therefore modest, but it shows up as faster and more stable learning.

6.3 EC3 — SARSA vs Q-Learning

What we did. SARSA [5] learns from the action the agent will really take next, a′ (chosen ε-greedily), instead of the best action. Both methods used $\alpha = 0.5$ and $\gamma = 0.99$, but $\varepsilon_0 = 0.9$, so that exploration lasts long enough for the two methods to behave differently.

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)] \tag{8}$$

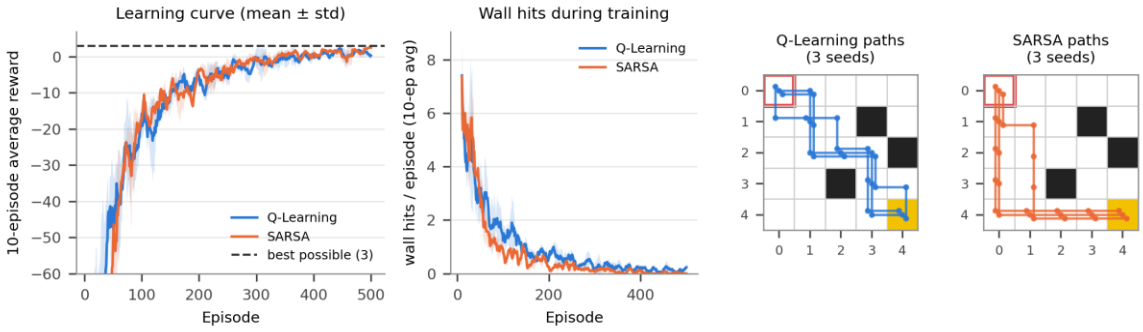


Figure 16. SARSA vs Q-Learning. From left: learning curves, wall hits per episode during training (both mean ± std over 3 seeds), and the final greedy paths of the 3 seeds for each method.

Table 15. SARSA vs Q-Learning ($\alpha = 0.5$, $\gamma = 0.99$, $\varepsilon_0 = 0.9$; mean ± std over 3 seeds).

Metric (3 seeds)	Q-Learning	SARSA
Episodes to learn	323.3 ± 22.4	354.0 ± 9.4
Average reward, last 50 episodes	0.87 ± 0.07	1.50 ± 0.34
Wall hits during training (500 episodes)	508 ± 84	395 ± 15
Wall-adjacent cells on the greedy path	3.7 ± 0.5	1.3 ± 0.5
Mean max Q in cells next to a wall	6.23 ± 0.31	-3.91 ± 1.55
Mean max Q in the other cells	5.08 ± 0.05	-0.58 ± 0.49
Greedy test: seeds reaching the goal	3/3 (8 moves)	3/3 (8 moves)

Result. SARSA hit walls less during training (395 vs 508 hits) and earned a higher reward in the last 50 episodes (1.50 vs 0.87), while learning about as fast. Both final paths have 8 moves, but SARSA's paths keep away from the walls: two of them follow the left and bottom edges, and on average they pass 1.3 cells next to a wall, compared with 3.7 for Q-Learning, whose paths cut through the middle.

Why SARSA is more careful near walls. Q-Learning assumes that from the next step on it will always act perfectly, so a cell next to a wall looks just as good as any other cell on a shortest path. SARSA learns from what it really does, including its random moves. Next to a wall a random move can cost −10, so SARSA gives those cells low values (mean best Q-value -3.9 vs 6.2 for Q-Learning) and prefers routes where a slip is cheap. In short, Q-Learning learns the best route for a perfect agent, while SARSA learns the safest good route for the agent it actually is.

6.4 EC4 — Epsilon schedules

What we did. Starting from $\varepsilon_0 = 0.9$ (other settings best), we compared three ways of shrinking ε for both methods: ×0.995 per episode (the default), ×0.98 per episode (faster), and a straight-line drop to 0.01 over 200 episodes.

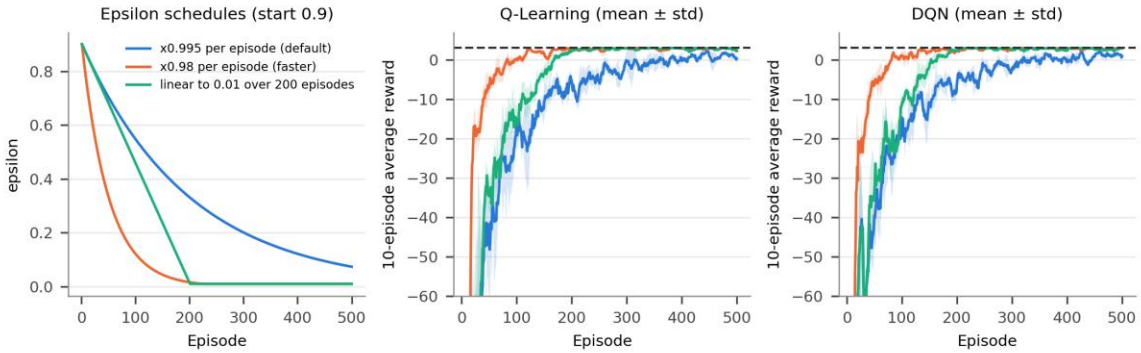


Figure 17. Epsilon schedules. Left: ε per episode. Middle and right: learning curves for Q-Learning and DQN (mean ± std over 3 seeds; dashed line: best possible reward).

Table 16. Effect of the epsilon schedule (mean ± std over 3 seeds).

Schedule ($\varepsilon_0 = 0.9$)	Episodes to learn	Avg reward, last 50 ep.	Time (s)	Greedy test: goal
Q-Learning: ×0.995 (default)	323.3 ± 22.4	0.87 ± 0.07	0.06 ± 0.00	3/3 (8 moves)
Q-Learning: ×0.98 (faster)	118.3 ± 0.9	2.75 ± 0.02	0.04 ± 0.00	3/3 (8 moves)
Q-Learning: linear over 200 episodes	181.0 ± 11.4	2.75 ± 0.02	0.05 ± 0.00	3/3 (8 moves)
DQN: ×0.995 (default)	313.3 ± 16.8	1.27 ± 0.12	6.60 ± 0.14	3/3 (8 moves)
DQN: ×0.98 (faster)	85.7 ± 2.4	2.86 ± 0.11	4.66 ± 0.08	3/3 (8 moves)
DQN: linear over 200 episodes	171.0 ± 12.6	2.71 ± 0.23	5.75 ± 0.18	3/3 (8 moves)

Result. The faster ×0.98 schedule was best for both methods: Q-Learning learned in 118.3 episodes instead of 323.3, and DQN in 85.7 instead of 313.3, with last-50 rewards close to the best. The linear schedule came second (181.0 and 171.0 episodes), and it was also faster in wall-clock time than the default.

What it shows. What matters most is how quickly ε falls in the first few dozen episodes: after 50 episodes ε is 0.33 with ×0.98, but still 0.68 with the linear schedule and 0.70 with the default, which never gets below 0.07 in 500 episodes and keeps paying for random moves. On this small maze less exploration is better, which agrees with Sections 5.3 and 5.6.

6.5 EC5 — Bigger maze (scaling test)

What we did. Inside extra_credit.py only, we built a 10×10 maze with 30 random walls (fixed seed 11, giving 26 distinct wall cells), start (0, 0) and goal (9, 9). BFS confirms that the goal can be reached: the shortest path has 18 moves, so the best reward is -7. Both methods used their best settings from Section 5.10 for 600 episodes; the DQN input becomes a one-hot vector of length 100.

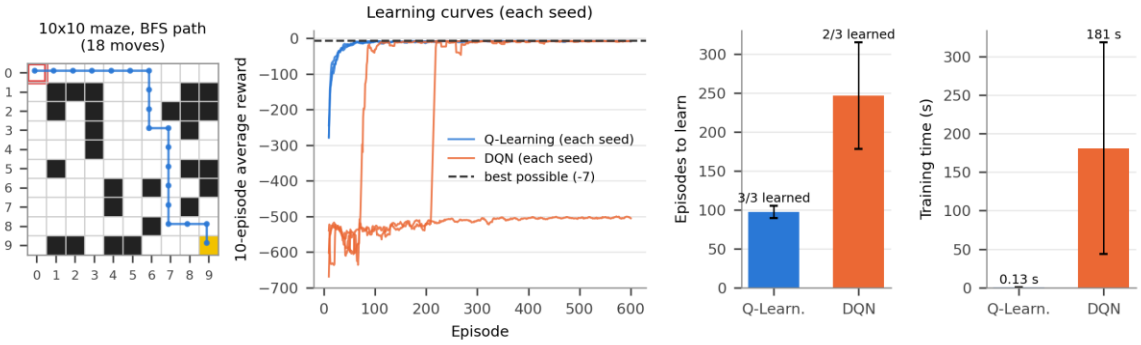


Figure 18. Scaling test on the 10×10 maze. From left: the maze with a BFS shortest path, learning curves of each seed (y-axis cut at −700), episodes to learn (seeds that learned) and training time.

Table 17. Q-Learning vs DQN on the 10×10 maze (best reward -7; mean ± std over 3 seeds).

Metric (3 seeds, 600 episodes)	Q-Learning	DQN
Seeds that learned	3/3	2/3
Episodes to learn (seeds that learned)	97.0 ± 7.9	246.5 ± 68.5
Average reward, last 50 episodes	-7.42 ± 0.09	-173.12 ± 233.87
Training time (s)	0.13 ± 0.00	180.9 ± 137.4
Environment steps	13823 ± 159	151870 ± 107952
Share of cells with an optimal greedy action	0.82 ± 0.04	0.67 ± 0.29
Greedy test: seeds reaching the goal	3/3 (18 moves)	2/3 (18 moves)

Result. Q-Learning learned on every seed in 97.0 episodes and needed only 0.13 s. DQN learned on 2 of 3 seeds (246.5 episodes for those) and needed 181 s on average, more than 1,400× longer. On seed 1 the DQN never reached the goal at all: 299,400 steps = 600 episodes × 499 moves, so every episode ran to the move limit.

Which method handles bigger mazes better, and why. In this test the table is clearly better. With one-hot inputs the network cannot share anything between neighbouring cells, so it must learn all 100 cells one by one, just like the table, but with slow, approximate gradient steps instead of exact updates. Exploration also differs: the zero table with random tie-breaking keeps trying new actions on its own, while an untrained network with ε = 0.1 can repeat the same bad moves for whole episodes and, in a large maze, may never find the goal. DQN becomes the better choice only when the state space is too large to store in a table and the input lets the network generalize, for example (x, y) coordinates or an image of the maze; a table cannot do that at all.

6.6 EC6 — Learning curves with error bands

What we did. We redrew the main Q-Learning vs DQN comparison (best settings) as the mean of the 3 seeds with a shaded ±1 standard deviation band.

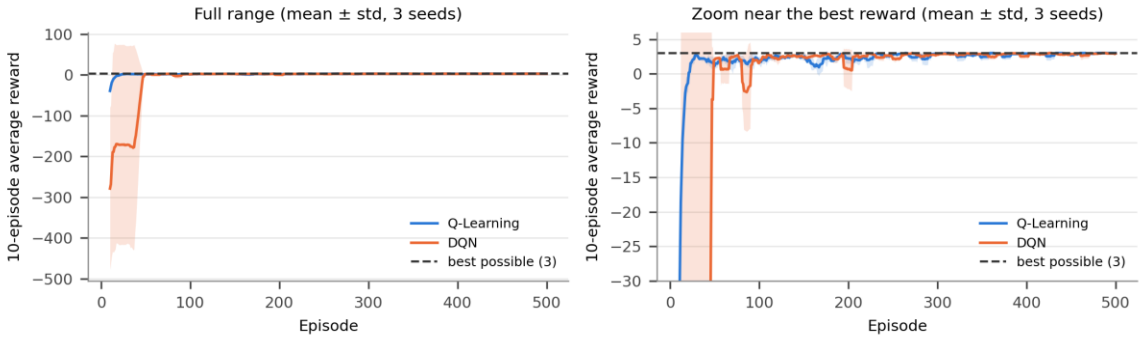


Figure 19. Q-Learning vs DQN, mean ± std over 3 seeds (10-episode average reward). Left: full range; right: zoom near the best reward.

Result. Q-Learning's band is narrow: all seeds learned within 23–28 episodes. DQN's band is very wide during the first ~50 episodes because seed 0 got stuck in long episodes; after that both methods stay close to the best reward (last-50 rewards 2.98 ± 0.02 and 2.87 ± 0.10). **What it shows.** A single run can be misleading; averaging several seeds and showing the spread tells both what usually happens and how reliable it is.

7. Conclusion

Both Q-Learning and the DQN learned an optimal 8-move route through the maze, and both trained agents reached the goal with the best possible reward of 3. Fixing the shared-weight bug gave the DQN a real target network. The best settings were **Q-Learning: $\alpha = 0.5$, $\gamma = 0.99$, $\varepsilon_0 = 0.1$** (25.3 ± 2.1 episodes to learn) and **DQN: learning rate 1e-3, $\gamma = 0.99$, $\varepsilon_0 = 0.1$, batch 64, C = 10, hidden size 64** (27.0 ± 17.0 episodes). The starting exploration rate was the most important setting for both methods, the learning rate came second, and the other settings mattered little on this maze. On this small, deterministic problem the table was faster, more consistent and about 106× cheaper per step, while the network's advantages (generalization, fixed size) only pay off on larger problems.

Limits of this study. Only one small maze was used (with 3, not 4, distinct walls), only 3 seeds per setting, and settings were changed one at a time, so interactions were checked only for the two most promising combinations. The last-50 reward includes random exploratory moves, and times were measured with four runs in parallel on a laptop. With one-hot inputs the DQN cannot share what it learns between neighbouring cells.

Future work. More seeds and a joint (grid or random) search over the settings; larger and randomly generated mazes; giving the DQN (x, y) coordinates or an image of the maze as input so it can generalize; and further DQN improvements such as Double DQN (Section 6), prioritized replay and dueling networks.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[3] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.

[4] H. van Hasselt, A. Guez and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.

[5] G. A. Rummery and M. Niranjan, “On-line Q-learning using connectionist systems,” Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department, 1994.